

ParallEdge: Exploiting Computing-Mobility Parallelism for Efficient 5G/6G Edge Computing

Rong Cong , Zhiwei Zhao , *Member, IEEE*, Linyuanqi Zhang , and Geyong Min , *Member, IEEE*

Abstract—With the emergence of the 5G/6G communications, edge computing has attracted increasing research interests in recent years. To provide pervasive 5G/6G edge computing services, numerous edge servers are required for service coverage, and the deployment cost can be >10000 times larger than the deployment cost of the 4 G infrastructure. To address this fundamental limit, we propose ParallEdge, a deployment scheme that employs mobile edge servers for cost-effective service coverage. ParallEdge is designed based on the observation that the processing delay and server moving delay become comparable in many computing-intensive applications in the 5G/6G edge computing. Unlike the traditional “move-then-process” frameworks, ParallEdge follows a “move-while-processing” paradigm, which exploits the parallelism between task processing and server movement to enable resource sharing among more users. Moreover, with the joint optimization of path planning and task scheduling for multiple mobile servers, the deployment cost for service coverage can be further reduced. We analyze the approximation gap of the proposed algorithm, and conduct extensive simulation experiments based on the real-world application data, and the results show that ParallEdge can significantly reduce deployment cost and improve resource utilization compared to the state-of-the-art schemes.

Index Terms—5G/6G Edge computing, server deployment, path planning, task scheduling, mobile servers.

I. INTRODUCTION

THE proliferation of the Internet-of-Things (IoT) and 5G/6G communication technologies have motivated the rise of pervasive, data-intensive and delay-sensitive applications, such as video analytics, autonomous driving and augmented reality [1]. In order to support massive computing requests and provide privacy preserving services, edge computing has emerged as a promising paradigm which deploys edge servers in the proximity of various end devices in smart factories, smart communities, etc. [2], [3].

Manuscript received 23 November 2022; revised 12 April 2023; accepted 5 June 2023. Date of publication 8 June 2023; date of current version 4 April 2024. This work was supported by the National Natural Science Foundation of China under Grants 61972074 and 61972075, in part by the Natural Science Foundation of Sichuan Province under Grant 2022NSFSC0885, and in part by the National Key Research and Development Program of China under Grant 2020YFE0200500. Recommended for acceptance by M. Li. (Corresponding authors: Zhiwei Zhao; Geyong Min.)

Rong Cong, Zhiwei Zhao, and Linyuanqi Zhang are with the College of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan 610051, China (e-mail: congrong@mobinets.org; zhiwei@mobinets.org; yuanqi@mobinets.org).

Geyong Min is with the College of Engineering, Mathematics, and Physical Sciences, University of Exeter, EX4 4PY Exeter, U.K. (e-mail: G.Min@exeter.ac.uk).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2023.3284217>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2023.3284217

One of the major obstacles faced by the 5G/6G edge computing is the tremendous deployment cost, i.e., a massive number of edge servers are required to provide large-scale service coverage. For example, the typical communication radius of the 5G base stations is 100-300 meters [4]. In the upcoming era of the 6G, the communication radius of the 6G base stations is a few tens of meters [5], [6]. Compared to that of the 4 G communications (1-25 kilometres) [7], the deployment cost for the same coverage can be drastically increased by 100-62500 times in the 5G/6G edge computing. In the envisioned multi-access edge computing, ultra-dense server deployment is needed to provide satisfactory Quality-of-Service (QoS) for end devices [4]. Thus, the practical deployment cost could be even higher.

Most of the existing work deploy static edge servers at fixed locations following the ETSI standards [8], [9]. They aim to combine the features of computation demands, wireless dynamics, geographical information, etc., to find the best locations for placing the edge servers. While these work can reduce the deployment cost or improve the QoS to some extent, the number of deployed servers is still unacceptably large. In the 5G/6G networks, the above imperfection will be further highlighted due to a more limited communication range of servers.

Some recent efforts deploy mobile servers for the purpose of cost-effective coverage [10], [11], [12], [13]. These work follow a repetitive “move-then-process” paradigm, where each mobile edge server periodically moves to a planned spot, collects and processes the computing requests from IoT devices, and then moves on to the next spot when all tasks have been finished at the current spot. For example, the state-of-the-art work [11] used this paradigm and proposed an adaptively joint optimization algorithm for the resource allocation and path planning, aiming to minimize the overall movement delay of mobile servers.

The underlying assumption of the “move-then-process” paradigm is that the transmission-and-processing delay is much smaller than edge servers’ moving time. Specifically, the previous work assume that IoT tasks are lightweight and the processing delay can be neglected when being executed on edge servers. As a result, optimizing the overall delay is nearly equivalent to optimizing the servers’ moving delay.

However, we observe that this assumption no longer holds in the 5G/6G edge computing paradigm, i.e., the task transmission-and-processing delay and the servers’ moving time become comparable for many computing-intensive 5G/6G applications. For example, in the applications of High Definition (HD) video analysis for periodic pedestrian detection [1], the processing time of one-minute 4 K video (29.8 GB) is around 57.6 minutes

(960 ms per frame, 60fps) [1]. The transmission time is around 23.84 s/238 ms with the 5G/6G communication technologies (10 Gbps/1Tbps) [14]. Meanwhile, the time for a server to move 1 km is about 4 minutes (15 km/h). As a result, the delay of server movement is no longer the only bottleneck of the overall delay. The sojourn time at each spot can be considerably large if we continue to follow the “move-then-process” scheme in the 5G/6G edge computing.

Intuitively, the sojourn time is only necessary for task transmissions, instead of both transmission and processing. If we change “move-then-process” to “move-while-processing” paradigm, i.e., a server can move to the next spot right after it has collected all tasks from the current spot, even if the task processing is not finished, then the comparable delays of task processing and server movement can be paralleled. In this way, the number of edge servers and the overall delay could be significantly reduced (as analyzed in Section II). However, this is a non-trivial issue due to the following challenges. 1) In the “move-while-processing” paradigm, a task’s requesting, processing and result returning need to be separated. As a consequence, each spot may probably be visited twice. Such a problem cannot be directly solved by the algorithms for Travelling Salesman Problem (TSP). The multi-visit TSP model [15] seems promising. However, it requires given edge weights (movement cost) and vertex weights (task workload), which become varying due to different levels of computing-mobility parallelism in the new “move-while-processing” paradigm. 2) To reduce the deployment cost and improve resource utilization, we need to optimize both path planning and task scheduling for multiple mobile servers. The two problems are coupled with each other, i.e., path planning requires the processing delay with given task schedules while task scheduling requires moving delay with given path plans.

To address the above challenges, we propose ParallEdge, a hybrid deployment framework following the “move-while-processing” paradigm. First, we formulate the server deployment in ParallEdge as a joint optimization problem consisting of device-server allocation, server path planning and task scheduling, which incorporates variable edge/vertex weights caused by the computing-mobility parallelism. Second, we propose a two-stage algorithm, based on elitist genetic algorithm and Gibbs sampling, to decouple the sub-problems and figure out the near-optimal solutions in a polynomial time. The major contributions of this paper are summarized as follows.

- We propose ParallEdge, a hybrid framework with both static and mobile edge servers for the full coverage. ParallEdge exploits the computing-mobility parallelism of mobile servers to reduce the deployment cost and improve resource utilization in the 5G/6G edge networks.
- We propose a joint optimization model of the coupled problems, i.e., device-server allocation, path planning and task scheduling, to reduce the number of required edge servers and improves the resource utilization of each server. The problem is also proved to be NP-hard.
- We propose a two-stage optimization algorithm and analyze the approximation gap and convergence. The results reveal that the computational complexity of this optimization algorithm is $O(M \times N^2)$, where M is the

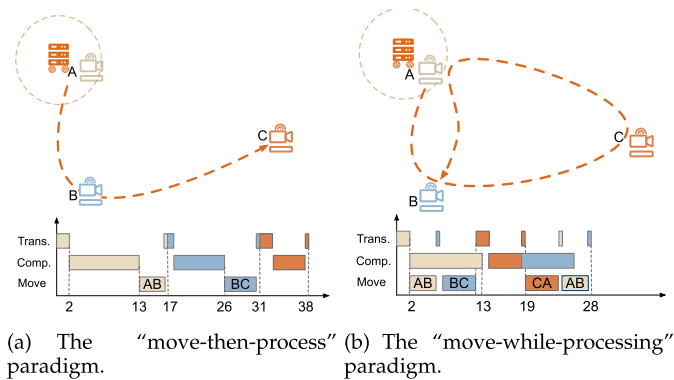


Fig. 1. Comparison between the existing mobile frameworks and ParallEdge. In the “move-while-processing” paradigm, the edge server can move to the next spot right after it has collected all requests from the current spot and then process the collected tasks during its movement.

number of edge servers and N is the number of IoT devices dispersed in the network.

- We compare ParallEdge with the state-of-art schemes via extensive simulation experiments based on three real-world applications. The results show that ParallEdge can significantly decrease the server deployment cost and improve resource utilization in the 5G/6G edge computing.

II. MOTIVATION

In this section, we use an atomic example to present the motivation, basic idea and challenges of ParallEdge.

Fig. 1 shows an example consisting of three camera-equipped IoT devices (A, B and C) with video analysis demands, e.g., traffic monitoring. These devices continuously capture HD videos and require analysis results every six hours. Based on the results, IoT devices can adjust the control policy of traffic lights or count crowds [1], [16].

We want to provide edge computing service for the three devices by deploying edge servers. If static edge servers are deployed [8], [9], each spot requires a separate edge server due to the limited 5G/6G communication range, and three servers are required in total. If we use mobile edge servers, only one server is enough as it can move across the three devices, thus the deployment cost is reduced. The existing frameworks using mobile servers [10], [11] follow the “move-then-process” paradigm, i.e., each server moves to a spot, collects and processes the tasks and then moves to the next spot. Fig. 1(a) illustrates the “move-then-process” paradigm. The server moves to device A and processes all tasks from A. After that, it moves to devices B and C. The timeline for the server is depicted in the lower part of Fig. 1(a). We see that 38 time units are used, among which the transmission and processing time takes 29 time units and server moving takes 9 time units.

Considering that the transmission and processing time cannot be reduced with the given tasks and edge server resources, the optimization space lies in the duration of server movement. Intuitively, if we allow “move-while-processing” and “hide” the moving time in the task duration, the total delay would be minimized. Fig. 1(b) shows the case where the server is able to move to the next spot while processing tasks from its previously

visited devices. As shown in the timeline, the server moves from A to B and C, and then moves back to A and B to return the processing results. Although the moving time is increased, the moving and processing are fully paralleled and the total delay is reduced to 28 time units.

From the example, we identify that: for data/computing-intensive and delay-tolerant applications in the 5G/6G edge computing, the “move-while-processing” paradigm can achieve significant improvement over “move-then-process” paradigm. Due to the higher transmission rate of the 5G/6G and wider applications of HD cameras [1], [14], [16], such applications become very common for supporting traffic monitoring [17], pedestrian detection [1], [16], HD video forensics [18]. *Our basic idea is to employ the “move-while-processing” paradigm to reduce the deployment cost, especially for the data/computing-intensive and delay-tolerant applications.* By finding out the appropriate paths and task schedules for the mobile servers, the movement and processing can be better paralleled and thus the overall deployment cost can be reduced. It is also worth noting that our work is compatible with real-time applications where the parallelism between moving and processing is limited. In such cases, the policy essentially becomes similar to the “move-then-process” or static policies.

However, it is a non-trivial task to realize the “move-while-processing” paradigm due to the following challenges. 1) In order to enable the parallelism between computing and movement, a task’s requesting, processing and result returning need to be separated, leading to that each spot requires a second visit. Such a problem cannot be easily characterized by the commonly used TSP model. Although the multi-visit TSP model [15] is promising, it requires the known edge weights (movement cost) and vertex weights (task workloads), which become varying in the “move-while-processing” paradigm due to different levels of computing-mobility parallelism. 2) To reduce the deployment cost and total delay, we need to determine both path planning and task scheduling for multiple servers. The two problems are coupled with each other: path planning requires the processing delay with given task schedules while task scheduling requires moving delay with given path plans.

The key idea of ParalleEdge is to exploit the computing-mobility parallelism to reduce the number of edge servers as well as the overall task delay. To address the above challenges, we formulate a joint optimization problem of path planning and task scheduling for multiple mobile edge servers, which incorporates variable edge/vertex weights caused by the paralleled computing and mobility. We further propose a two-stage optimization algorithm with low complexity. We also analyze the approximation gap and convergence of the proposed algorithm in Section IV. Next, we present the detailed design of ParalleEdge in Section III.

III. DESIGN OF PARALLEEDGE

In this section, we present the system model and the main design of ParalleEdge. For ease of reference, the key notations used in the paper are summarized in Table I.

TABLE I
SUMMARY OF KEY NOTATIONS

Notation	Description
$\mathcal{N}$	Set of IoT devices in the area.
$\mathcal{M}$	Set of edge servers.
$\mathcal{N}_j$	Set of devices allocated to the server j .
c_i	Computation demand of device i .
b_i	Input data size of device i .
d_i	Output data size of device i .
ddl_i	Latency requirement of device i ’s task.
F	Computation frequency of edge server.
S	Storage capacity of edge server.
V	Movement speed of mobile server.
A	Device-server allocation.
$\mathbf{P}$	Paths of all servers.
O	Processing schedule of tasks.
$t_{j,2 \mathcal{N}_j }^L$	The overall delay of server j .
t_i^R	The time when device i receives the results.
$p_{j,i}$	The i -th spot in server j ’s path.
$t_{j,i}^A$	The time when server j arrives at $p_{j,i}$.
$t_{i,j}^F$	The time when server j finishes task i .

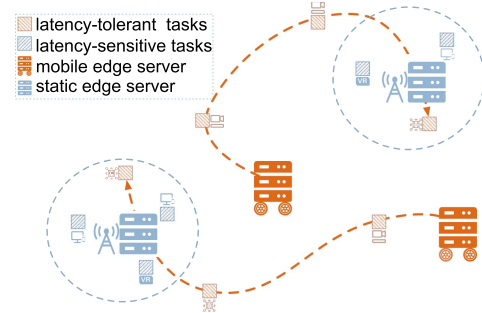


Fig. 2. Architecture of ParalleEdge.

A. System Model

We consider deploying a 5G/6G edge network to provide computing service for IoT devices in a given area such as smart factory [19] and smart communities [2]. The IoT devices are placed at the points of interest (PoIs) in the area and continuously generate computing-intensive tasks to be processed by edge servers (as shown in Fig. 2). The tasks that invoke edge services include delay-sensitive tasks like AR/VR and real-time tasks like monitoring and forensics [1], [18].

The basic idea of ParalleEdge is to provide edge service to cover all the IoT devices with both stationary and mobile edge servers. For mobile servers, we follow “move-while-processing” paradigm to exploit the parallelism between server moving and task processing. For delay-tolerant tasks, as analyzed in Section II, server-moving delay and task-processing delay are comparable and more delay could be saved by the parallelism. For real-time applications, there may not be much chance for the parallelism. We do not explicitly define which devices are delay-sensitive or delay-tolerant, instead, we input all IoT requests into our algorithm (Section IV-C). The resulting strategy may consist of some servers that move nowhere (for the real-time

applications), and these servers naturally become stationary servers.

Next, we describe the system model of Paralledge. The mobile edge servers, indexed as $\mathcal{M}$, are standardized edge servers that can move with the help of vehicles, robots, etc. [10], [11]. A mobile server is described using a tuple $\{F, S, V\}$, where F is the computation resource (e.g., CPU frequency), V is the average movement speed, and S is the storage capacity of the server. Obviously, the limited capacity of storage resource prevents a mobile server from collecting massive requests and computing them while moving. It needs to return some results to release enough storage for other devices.

IoT devices are dispersed in the network, indexed as $\mathcal{N}$. A task from device i located in l_i can be described as a tuple $(l_i, c_i, b_i, d_i, ddl_i)$, where c_i is the computation demand (e.g., total number of CPU cycles required [20]), b_i is the data size of the task inputs, d_i is the data size of the task results, and ddl_i is the latency requirement of the task (e.g., the results of video monitoring should be returned within a periodic deadline). Following the existing studies [8], [21], the centroid of each communication intersection areas is treated as a candidate visiting position u . The distance between candidate visiting position u and IoT device v , denoted as $dis(u, v)$, can be calculated by Euclidean distance or the navigation distance (which is available in many open map apps like graphhopper¹ and OSRM²).

B. Problem Statement and Formulation

Overview: The objective of Paralledge is to minimize the deployment cost, i.e., the number of required edge servers. Since it is a non-trivial task, we use a step-by-step manner to find the minimum number. Specifically, we start with the ideally minimum server number, i.e., $\lceil \frac{\mathbb{T}}{\mathbb{R}} \rceil$, where $\mathbb{T}$ denotes the overall demands of tasks and $\mathbb{R}$ denotes the resource of server.³ For this number of servers, we try to figure out the optimal decision of task allocation, path planning and task scheduling by solving the modelled optimization problem. If the number of servers is too small to find a feasible solution to the problem, we increase the number and continue to solve the problem. When a solution is found, both the server number and the deployment strategy are determined. The rationale behind this workflow is that if the task deadlines cannot be met with the minimum overall delay, the given number of servers should be insufficient. Due to the NP-hardness of the problem (as proven in Theorem 1), our scheme is not optimal yet still achieves significant improvement over the existing works (as evaluated in Section V). We also propose an algorithm with reduced complexity for practical use of Paralledge (as described in Section IV).

The Joint Optimization Model: Given the number of deployed servers, our goal is to minimize the overall delay of servers by optimizing the following coupled sub-problems: 1) device-server allocation (A), 2) paths of all servers (P), and 3) processing schedule of tasks (O). Device-server allocation results impact

the solution space of other sub-problems, while the evaluation of the allocation policy relies on the optimization results of other sub-problems. Meanwhile, path planning requires the processing latency determined by task scheduling, while task scheduling requires the processing schedule of tasks which is in turn determined by the planned paths. Given the number of deployed servers, the optimization problem can be described as follows.

$$\begin{aligned}
 \mathcal{P} : & \min_{A, P, O} \sum_{j \in \mathcal{M}} t_{j, 2|\mathcal{N}_j|}^L & (1) \\
 \text{s.t. } \mathcal{C}_1 : & S(p_{j,i}) \leq S_j, \quad \forall j \in \mathcal{M} \\
 \mathcal{C}_2 : & t_i^R \leq ddl_i, \quad \forall i \in \mathcal{N} \\
 \mathcal{C}_3 : & \sum_{j \in \mathcal{M}} a_{j,i} = 1, \quad \forall i \in \mathcal{N} \\
 \mathcal{C}_4 : & a_{j,i} \in \{0, 1\}, \quad \forall j \in \mathcal{M}, i \in \mathcal{N} \\
 \mathcal{C}_5 : & n_{i,j} \leq 2, \quad \forall j \in \mathcal{M}, i \in \mathcal{N}_j \\
 \mathcal{C}_6 : & o_{j,i} \in \{1, 2, \dots, |\mathcal{N}|\} \quad \forall j \in \mathcal{M}, i \in \mathcal{N}_j & (2)
 \end{aligned}$$

We formulate a Joint Optimization problem of Path planning and Task scheduling for multiple mobile edge servers (*JOPT*), aiming to minimize the overall delay of all servers, as shown in (1). The overall delay of server j is $t_{j, 2|\mathcal{N}_j|}^L$, i.e., the time duration from the server working to finishing its allocated tasks $\mathcal{N}_j$. Hence, the overall delay of all servers can be represented as $\sum_{j \in \mathcal{M}} t_{j, 2|\mathcal{N}_j|}^L$. Constraint $\mathcal{C}_1$ ensures that the sum of collected computing requests cannot exceed the storage capacity of servers during the movement process. Constraint $\mathcal{C}_2$ ensures that the task processing results should be returned within the task deadline ddl_i , where t_i^R denotes the time when device i receives the results. Constraints $\mathcal{C}_3$ and $\mathcal{C}_4$ indicate that one and only one device can be allocated to server j . Constraint $\mathcal{C}_5$ ensures that each mobile server j can move to one IoT device i at most twice, where $n_{i,j}$ represents the number of server j 's visits to device i . The last constraint $\mathcal{C}_6$ indicates the priority order of task processing.

Theorem 1: Problem JOPT is NP-hard.

Proof: Recall that Problem JOPT consisted of three sub-problems: device-server allocation, path planning and task scheduling. We will analyze the hardness of sub-problems in turn and derive the hardness of the whole problem JOPT.

Device-server allocation: This sub-problem can be reduced to a classical NP-hard problem, the Set Covering Problem (SCP) [22]. Let $\mathcal{U} = \mathcal{N}$ be the set of IoT devices. Given task demands and server configuration, the collection of set $\mathcal{S}$ can be constructed as follows. First, we figure out all combinations of IoT devices in the area. The i -th combination is denoted as $\mathcal{S}_i$. Then, for each combination of devices, we add the combination $\mathcal{S}_i$ to $\mathcal{S}$ if the requirements of these devices can be satisfied by one mobile server. It is easy to see that this sub-problem is equivalent to finding the smallest sub-collection of $\mathcal{S}_i \in \mathcal{S}$ whose union equals the universe set $\mathcal{U}$.

Path planning: This sub-problem can be reduced to an NP-hard problem, the Many-Visit Travelling Salesman Problem (MV-TSP) [15], where the salesman (mobile server) needs to

¹[Online]. Available: <https://github.com/apneadiving/Google-Maps-for-Rails>

²[Online]. Available: <https://github.com/Project-OSRM/osrm-backend>

³More details of the calculation of the ideally number of servers are described in Section IV-C.

find an optimal tour with minimal travelling distance and visit each city (IoT device) two exactly times.⁴

Task scheduling: This sub-problem is an integer programming problem according to the domain of O , which is not NP-hard. However, we cannot directly leverage the existing scheduling methods [23], [24], like earliest-deadline-first [23] and energy-greedy [24], to solve our task scheduling problem. Because the existing scheduling methods view the whole tasks as the available tasks, while the available tasks in JOPT refers to the tasks that have been offloaded to the server, which depends on the path planning results.

Although the sub-problem task scheduling is not NP-hard, the overall optimization problem JOPT is still NP-hard, due to the NP-hardness of device-server allocation and path planing and the coupled relationship among the three sub-problems. Hence, problem JOPT is NP-hard. $\square$

C. Modelling Decision Variables A , $\mathbf{P}$, and O

Now we model the decision variables in JOPT.

1) **Device-Server Allocation (A):** We denote the matching relationship between devices and servers using a set of boolean flags as follows.

$$A = \{a_{j,i} \in \{0, 1\} : j \in \mathcal{M}, i \in \mathcal{N}\}, \quad (3)$$

where $a_{j,i}$ indicates whether the IoT device i is allocated to server j . Each device will be handled by one and only one mobile server, satisfying constraints $\mathcal{C}_3$ and $\mathcal{C}_4$. Using the allocation result, the set of IoT devices allocated to server j can be indicated as

$$\mathcal{N}_j(A) = \{i | a_{j,i} = 1, i \in \mathcal{N}\}. \quad (4)$$

2) **Planned Paths ($\mathbf{P}$):** Paths need to imply the “move-while-process” scheme of ParalleEdge, i.e., the different goals (request collection or result returning) when the server moves to the same position. The path of server j is denoted as:

$$\mathbf{P}_j = (p_{j,i} \in \{l_k\}_{k \in \mathcal{N}_j} : j \in \mathcal{M}, i \in \{1, 2, \dots, 2|\mathcal{N}_j|\}), \quad (5)$$

consisting of a sequence of visited locations, where $|\mathcal{N}_j|$ is the number of devices allocated to the server j , and $p_{j,i}$ is the i -th visited location in the path of server j . Given a pair of adjacent elements in path $\mathbf{P}_j$: $p_{j,u} = i$ and $p_{j,v} = i$ ($u < v$), there are two situations: 1) $u + 1 < v$, mobile server j will process tasks from device i while moving. 2) $u + 1 = v$, mobile server j will process tasks from device i in place. For example, the path of server in Fig. 1(b) is (A, B, C, C, A, B) . We leverage two elements in a path to indicate one device, so the length of $\mathbf{P}_j$ is $2|\mathcal{N}_j|$.

When server j visits $p_{j,i}$, its occupied storage $S(p_{j,i})$ can be computed as

$$S(p_{j,i}) = \sum_{k=1}^i b_k, \quad p_{j,k} = p_{j,l}, \quad k \leq i < l. \quad (6)$$

⁴Although path planning itself can be solved using the algorithms for MV-TSP, we cannot directly use them to solve JOPT due to the coupling of the three sub-problem.

3) **Task Schedules (O):** The computing-mobility parallelism results in more frequent situations where multiple tasks are stored in the server simultaneously. An appropriate processing schedule of tasks is crucial to utilize server resources (both computation and storage) efficiently. We use O to indicate the result of task scheduling.

$$O = \{o_{j,i} \in \{1, 2, \dots, |\mathcal{N}|\} : j \in \mathcal{M}, i \in \mathcal{N}\}, \quad (7)$$

where O_j is the scheduling result of tasks from $\mathcal{N}_j$, and $o_{j,i}$ is the priority of tasks from device i allocated to server j . The lower the value, the higher the priority.

D. Modelling Time Cost $t_{j,u}^L$

The overall delay of server j is the time when the server completes all assigned tasks $\mathcal{N}_j$ (determined by (4)), denoted as $t_{j,2|\mathcal{N}_j|}^L$.

For each position $p_{j,i}$ in path $\mathbf{P}_j$, the corresponding time can be modelled by three parts: arrival time $t_{j,i}^A$, leaving time $t_{j,i}^L$ and the stopping duration. Note that receiving time t_i^R is equal to the leaving time when the server revisits device i . Obviously, for each position in path $\mathbf{P}_j$, the arrival time of point $p_{j,i}$ can be obtained by its adjacent point $p_{j,i-1}$ using $t_{j,i}^L = t_{j,i-1}^A + \text{dis}(p_{j,i-1}, p_{j,i})/V$. Given the path where server j visits device i at point $p_{j,u}$ and $p_{j,v}$, i.e., $p_{j,u} = p_{j,v} = i$, the relationship among the three parts is

$$t_{j,u}^L(\mathbf{P}_j, O_j) = \begin{cases} t_{j,u}^A + T_{j,i}^{up} & \text{if } u < v \\ t_{j,u}^A + \max\{T_{j,i}^{down}, t_{j,k}^F + T_{j,i}^{com}\} & \text{if } u > v \end{cases} \quad (8)$$

Here, k is the index of the last finished device, obtained as $k(A, O_j) = \max\{l | a_{j,l} = 1, o_{j,l} < o_{j,i}, \forall i, l \in \mathcal{N}_j\}$.

The above equation also indicates the situations when server j visits device i .

- 1) The server *first* visits device i for request collection. In this case, the server stops to collect the requests. Hence, its stopping duration is the uploading time $T_{j,i}^{up}$, calculated as (9).
- 2) The server *second* visits device i for returning results. Note that if higher-priority tasks or itself are still unfinished, the server has to stay in place until finishing these tasks. In this case, the stopping duration is finish time of higher-priority task $t_{j,k}^F$ and computing time $T_{j,i}^{com}$ for residue tasks from device i . Otherwise, the server stops at $p_{j,u}$ just for downloading results. Hence, the stopping duration can be presented as $\max\{T_{j,i}^{down}, t_{j,k}^F + T_{j,i}^{com}\}$.

Transmission time the task from device i is denoted as $T_{j,i}^{up}$. It can be obtained by Shannon formulation with 5G/6G channel multiplexing (non-orthogonal multiple access in 5G [25] and orbital angular momentum multiplexing in 6G [5]).

$$T_{j,i}^{up} = \frac{b_i}{B \log_2 \left(1 + \frac{G_{i,j}}{\omega_0 + \sum_{k \in \mathcal{N} \setminus i} G_{k,j}} \right)}, \quad (9)$$

where B indicates the wireless bandwidth, ω_0 denotes the background noise power and $G_{i,j}$ is the channel gain from IoT device i to the server j . Similarly, the downloading time can be

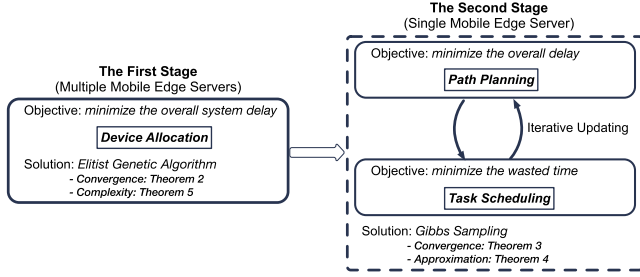


Fig. 3. Illustration of the two-stage optimization algorithm.

obtained as

$$T_{j,i}^{down} = \frac{d_i}{\text{Blog}_2 \left(1 + \frac{G_{j,i}}{\omega_0 + \sum_{k \in \mathcal{N} \setminus i} G_{j,k}} \right)}. \quad (10)$$

Computing time of tasks from device i assigned to the server j is denoted as $T_{j,i}^{com}$, which can be calculated by $T_{j,i}^{com} = \frac{c_i}{F}$. For the simplicity of problem, we assume that CPU frequency of server is static, as many studies [11], [26].

IV. TWO-STAGE OPTIMIZATION ALGORITHM

According to Theorem 1, JOPT is an NP-hard problem. To solve the JOPT problem with an acceptable complexity, we design a two-stage optimization algorithm that optimizes the multi-server scenarios and single-server scenarios in turn, as shown in Fig. 3. Leveraging both the elitist genetic algorithm and Gibbs sampling, our algorithm achieves a polynomial complexity $O(M \times N^2)$, where M is the number of deployed servers and N is the number of IoT devices in the target area.

A. The First Stage of Algorithm: Multi-Server Scenarios

For multi-server scenarios, the first thing to realize ParalleEdge is to allocate IoT devices to each server. Then the multi-server optimization problem can be transformed into a single-server optimization problem. However, we cannot directly allocate devices by partitioning the target area like existing multi-server works following the “move-then-process” paradigm [10], [12]. Because, the implicit logic behind this idea is that minimum moving distance would lead to the minimum overall delay, which does not hold in our “move-while-processing” paradigm, where the server moving process may be an opportunity to improve the computing-mobility parallelism and thus reduce the overall delay. Therefore, We need to design a novel device allocation scheme for ParalleEdge.

Recall that problem JOPT is a non-convex problem with multiple local optimal solutions, due to the integer variables A , P , O and its non-convex optimized objective $\sum_{j \in \mathcal{M}} t_{j,2|\mathcal{N}_j|}^L$. As a result, the devised algorithm needs to have the ability to escape local optimal solutions of JOPT with low computing complexity.

Genetic Algorithms (GA) are empirically found to provide global near-optimal solutions, based on the principles of natural biological evolution. Mutation operations in GA avoid the algorithm fall into local optimal. Elitist Genetic Algorithm (EGA), a varied type of GA, has a higher convergence speed than GA by saving the best individual to the next generation.

Algorithm 1: The First Stage of Algorithm.

Input: IoT device set $\mathcal{N}$, Mobile edge server set $\mathcal{M}$

Output: Device allocation result $\mathbf{A}$

- 1: $Iter \leftarrow 1$
- 2: Initialize population $POP(1)$ with N_P chromosomes, which satisfy the constraints of Hamming distance and are feasible for JOPT.
- 3: **for** $Iter = 1, 2, \dots, Iter_{EGA}$ **do**
- 4: Calculate $Fit(\cdot)$ of each chromosome in $POP(Iter)$
- 5: Record chromosome C_i with the highest fitness
- 6: Using tournament selection to select N_P parents
- 7: Apply crossover to generate $N_P - 1$ children
- 8: Apply mutation to newly generated children
- 9: $POP(Iter + 1) \leftarrow POP(Iter + 1)' \cup C_i$
- 10: **end for**

Based on EGA, we design the first stage of our algorithm to transform the multi-server deployment problem to the single-server deployment problem. The first stage of the two-stage optimization algorithm consists of three parts: initialization (lines 1 to 2), generation evolution (lines 3 to 10) and elitist reserve (lines 5 and 9), as shown in Algorithm 1. In each iteration of evolution, the current generation executes fitness computation, crossover, selection and mutation operation in turn, to generate efficient genes of the next iteration.

Realizing the EGA on our algorithm still faces two challenges.

- 1) Traditional chromosome representations, like 0-1 structure, are not suit for the device-server allocation sub-problem in JOPT.
- 2) An objective-oriented fitness function cannot guarantee the task deadlines in JOPT, thus we need to design a fitness function that trades off the optimization objective and device QoS constraints. To address the above challenges, we first propose an array structure of chromosomes to represent allocation solutions, where the gene is the index of IoT device and a chromosome consists of M genetic segments with size $|\mathcal{N}_j|$. If a gene i is located in genetic segment S_j , we indicate that device i is allocated to server j . Although this chromosome structure can effectively represent the device-server allocation results, it also introduces the duplicate allocation problem, i.e., one IoT device would be allocated to multiple servers violating Constraint C_3 and C_4 . To deal with the duplication problem, in crossover operation, we design an one-to-one mapping scheme between two parents. For example, in Fig. 4, we build two crossover maps: device 1 to device 2 and device 7 to device 8, meaning that the devices out of the maps cannot be re-allocated. Second, we propose a fitness function $Fit(\cdot)$ to evaluate the quality of allocation policy, which unites the optimization objective of JOPT and QoS constraints of tasks, given as

$$Fit(C_k) = \underbrace{- \sum_j t_{j,2|\mathcal{N}_j|}^L(A, \mathbf{P}, C_k)}_{\text{optimization objective}} + \underbrace{\sum_{i: t_{j,i}^L > ddl_i} (t_{j,i}^L - ddl_i)}_{\text{QoS constraint}}.$$

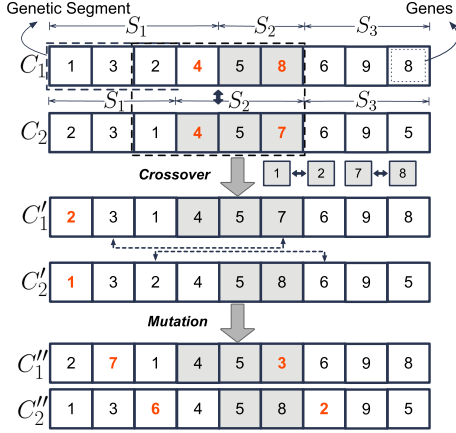


Fig. 4. Illustration of Algorithm 1.

Algorithm 2: The Second Stage of Algorithm.

Input: Set of IoT devices $\mathcal{N}$, Set of mobile servers $\mathcal{M}$, Result of device allocation A

Output: Paths $\mathbf{P}$, Processing schedule of task O

```

1: for all mobile server  $j$  in  $\mathcal{M}$  do
2:   According to  $A_j$ , get the devices  $\mathcal{N}_j$  assigned to  $j$ 
3:   for  $iter = 1, 2, \dots, Iter_{Gibbs}$  do
4:     Given  $O_j^{t-1}$ , obtain  $\hat{\mathbf{P}}_j^t$  using the 2-OPT
5:     Given  $\hat{\mathbf{P}}_j^t$ , obtain  $\hat{O}_j^t$  using Algorithm 3
6:     Compute  $\hat{t}_{j,2|\mathcal{N}_j}^L(\hat{\mathbf{P}}_j^t, \hat{O}_j^t)$ 
7:      $\rho \leftarrow \frac{1}{1 + \exp[-\omega(\hat{t}_{j,2|\mathcal{N}_j}^L - t_{j,2|\mathcal{N}_j}^L)]}$ 
8:     With the probability  $\rho$ , accept the new policy
        $\mathbf{P}_j^t \leftarrow \hat{\mathbf{P}}_j^t, O_j^t \leftarrow \hat{O}_j^t$ . Otherwise, keep the policy
       unchanged  $\mathbf{P}_j^t \leftarrow \mathbf{P}_j^{t-1}, O_j^t \leftarrow O_j^{t-1}$ 
9:   end for
10: end for

```

Now, we shall analyze the convergence of Algorithm 1.

Theorem 2: The first-stage of the proposed optimization algorithm (Algorithm 1) can converge to the optimal solution of device allocation in JOPT with any choice of initial population as iteration increases.

Proof: Please see Appendix A.

According to Theorem 2, the convergence of Algorithm 1 is affected by the value of its hyperparameter $Iter_{EGA}$. We further evaluate the impact of $Iter_{EGA}$ on algorithm convergence in Section V. Note that Theorem 2 only shows the algorithm convergence of device allocation. The algorithm convergence for the entire JOPT problem needs to be combined with the algorithm performance on the other sub-problems, which will be analyzed in Section IV-B.

B. The Second Stage of Algorithm: Single-Server Scenarios

Based on the results of device allocation, JOPT can be transformed into a single server scheduling problem, consisting of two coupled sub-problems: path planning and task scheduling,

Algorithm 3: Task Scheduling in Algorithm 2.

Input: Set of allocated IoT devices $\mathcal{N}_j$, Mobile edge server j , Movement path $\mathbf{P}_j$

Output: Processing schedule of task O_j

```

1:  $Que \leftarrow \{\}$ 
2: for all device  $i \in \mathcal{N}_j$  do
3:   Obtain  $t_{j,i}^F$  and  $t_{j,i}^A$ 
4:   if  $t_{j,i}^F > t_{j,i}^A$  then
5:     Add device  $i$  into  $Que$ 
6:   end if
7: end for
8: for all device  $i \in Que$  do
9:   repeat
10:     $o_{j,i} \leftarrow o_{j,i} - 1$ 
11:    Update  $t_{j,i'}^F$  and  $t_{j,i'}^A$  of each device  $i' \in \mathcal{N}_j$ 
12:  until  $t_{j,i}^F \leq t_{j,i}^A$  or  $o_{j,i} == 1$ 
13:  Delete device  $i$  in  $Que$ 
14: end for

```

i.e., the solution to one sub-problem requires the input of solution to another. To solve these sub-problems, we propose the second stage of our algorithm that iteratively updates the decisions of path planning and task scheduling based on Gibbs sampling, as shown in Algorithm 2.

According to the result of device-server allocation, we transform the multi-server problem into the single-server problem in the outermost loop (lines 1 to 10). Then, the algorithm exploits Gibbs sampling for each mobile server to update processing orders and movement paths iteratively (lines 3 to 9). Specifically, we optimize one decision with another decision unchanged (lines 4 to 5), and then associate the conditional probability distributions of two decisions with optimization objective using the acceptance probability ρ (lines 7 to 8).

The parameter $\omega \geq 0$ is a smoothing factor, used to control the exploitation-exploration proportion. When ω is large, the algorithm tends to explore more possible solutions. And when ω is small, it tends to exploit solutions with better performance. The selection of ω will be discussed in Section V. Given the processing schedule of tasks, path planning is solved by 2-OPT algorithm [27] with the path representation in ParalleEdge, i.e., (5).

Given movement paths, task scheduling can be solved using Algorithm 3. Specifically, we maintain queue Que to record tasks with optimization opportunity from task scheduling. For each device k , its computing-finish time $t_{j,i}^F$ and arrival time $t_{j,i}^A$ are obtained using (8)⁵ (lines 2 to 3). There are two situations when the server revisits one device: 1) the server has to stay in place waiting the completion of device's tasks ($t_{j,i}^F > t_{j,i}^A$), 2) tasks from this device have been completed before the server revisit ($t_{j,i}^F \leq t_{j,i}^A$). In the first case, significant time is wasted for waiting, as shown in the top part of Fig. 5. Thus, we propose the task scheduling strategy, in which the priorities of tasks in

⁵Here, parameter i is the index of path when server j revisits device k , satisfying $p_{j,i} = p_{j,i'} = k, i > i'$

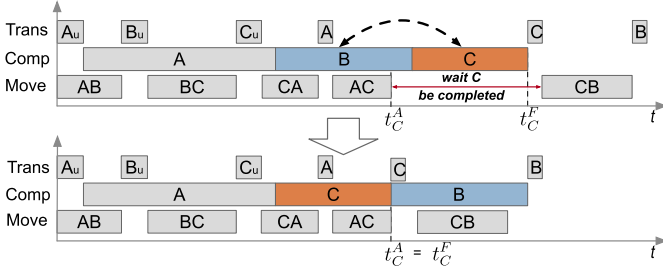


Fig. 5. Illustration of task scheduling in Algorithm 3.

this situation are increased to reduce the in-place waiting time of the server, as shown in Fig. 5.

To better comprehend the performance of Algorithm 2, we shall analyze its convergence and the approximation gap in Theorems 3 and 4.

Theorem 3: When $\omega \rightarrow \infty$, the second-stage of the proposed optimization algorithm (Algorithm 2) can converge to the minimum overall delay of a single server from any starting state of Markov chain.

Proof: Please refer to Appendix B. $\square$

Theorem 4: The approximation gap of Algorithm 2 is $\frac{1}{\omega} \log |\mathcal{X}_j|$, where $|\mathcal{X}_j|$ is the number of feasible management decisions (both paths and schedules) of mobile server j .

Proof: The goal of Algorithm 2 is to minimize the overall delay of a server, evolved as the sum of nodes' serving time. The serving time of a IoT node is composed of utilized time (computing time) and wasted time (non-parallel moving time and waiting time). For convenience of presentation, the serving time of device i with path $\mathbf{P}_j$ and task schedules O_j is denoted by $y_i(O_j, \mathbf{P}_j)$.

The overall delay of server j can be rewritten as

$$\min_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} y_i(O_j, \mathbf{P}_j). \quad (11)$$

This minimal problem can be transformed into the following maximal problem:

$$\max_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j). \quad (12)$$

Let $\max_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j)$ be the optimal solution of (12) and $\max_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j)$ be the expected solution solved by Algorithm 2. Leveraging the fact that the approximation gap between the max function $\max_{i=1, \dots, n} x_i$ and the convex *log-sum-exp* function is upper-bounded by $\frac{1}{\omega} \log \sum_{i=1}^n \exp(\omega x_i)$, $\frac{1}{\omega} \log n$ [28], we can derive

$$\begin{aligned} & \max_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j) \\ & \leq \frac{1}{\omega} \log \left(\sum_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \exp \left(\omega \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j) \right) \right) \end{aligned}$$

$$\leq \max_{(\mathbf{P}_j, O_j) \in \mathcal{X}_j} \sum_{i \in \mathcal{N}_j} -y_i(O_j, \mathbf{P}_j) + \frac{1}{\omega} \log |\mathcal{X}_j|. \quad (13)$$

The relaxation of (13) indicates the gap between the optimal solution and the approximated solution, i.e., the approximation gap of Algorithm 2. $\square$

We now analyze the computation complexity of the two-stage optimization algorithm.

Theorem 5: Given the edge network with IoT devices with N tasks and M mobile servers, the computation complexity of the Two-stage Optimization Algorithm is $O(M \times N^2)$.

Proof: The bottleneck of the two-stage optimization algorithm is computing the fitness function, in which Algorithm 2 is called for each segment. Algorithm 2 consists of two sub-algorithms: modified 2-OPT ($O(N^2)$) and the heuristic algorithm for task scheduling ($O(N)$). Considering that the number of chromosome N_P can be viewed as the constant parameter of the algorithm, the computation complexity is $O(N_P \times M \times N^2) = O(M \times N^2)$. $\square$

By combining the above theorems, we can see that the two-stage optimization algorithm can coverage to the approximately optimal solution of the JOPT problem with gap $\frac{1}{\omega} \log |\mathcal{X}_j|$ in a polynomial time.

C. Deployment Algorithm

The two-stage optimization algorithm only solves the scheduling of mobile servers, while the minimum number of servers and the corresponding deployment strategy are unknown.

Intuitively, we can start with the small number of servers as input and try to figure out the scheduling strategy of mobile servers. If the number of servers is too small to satisfy the QoS constraints of devices, we increase the number and continue to solve it. However, the execution time is unacceptable since the algorithm searches from a tiny number of servers far away from the optimal value. We expect to start with the close number of required servers, which can be obtained using Algorithm 2.

Specifically, we obtain this number using $\lceil \frac{|\mathcal{N}|}{|\mathcal{N}_s|} \rceil$, where $|\mathcal{N}|$ is the number of all IoT devices and $|\mathcal{N}_s|$ is the number of devices allocated to a single server. Now the goal is to find $|\mathcal{N}_s|$, i.e., the maximum number of devices within the ability of one server. Initially, we try to serve all devices using one server, i.e., $|\mathcal{N}_s| = |\mathcal{N}|$. If $|\mathcal{N}_s|$ is too large to find a feasible solution to the single-server scheduling problem using Algorithm 2, we reduce this number ($|\mathcal{N}_s| = |\mathcal{N}_s| - 1$) and randomly select $|\mathcal{N}_s|$ devices. When we find a solution without the violation of QoS, $|\mathcal{N}_s|$ is recorded as the maximum number of devices that one server can handle. Obviously, this number is just a near-maximum number, depending on the selection of devices. Hence, we execute the above steps for 100 times and view the average result as the near-maximum number. As a result, the close-floor number of servers can be estimated.

Note that we do not design another algorithm for stationary server deployment in ParalleEdge, since it can be driven by the results of two-stage optimization algorithm. If a server is

TABLE II
SETTINGS OF KEY PARAMETER IN THE SIMULATION

Para.	Setting	Description
N	300	Number of IoT devices.
F	3.0 GHz	CPU frequency of server.
S	5 TB	Storage capacity of server.
V	30 km/h	Movement speed of mobile server.
ddl	U[1,120]	Delay requirement of task.
$Iter_{EGA}$	200	Iteration number of Algorithm 1.
$Iter_{Gibbs}$	150	Iteration number of Algorithm 2.
ω	50	Smoothing factor in Algorithm 2.

scheduled to move nowhere, it naturally deployed as a stationary edge server.⁶

V. SIMULATION ANALYSIS

In this section, we conduct extensive simulation experiments to evaluate the performance of ParallEdge and compare it with the state-of-art edge frameworks about server deployment.

A. Setup and Benchmarks

Experiment Settings: We simulate a $10\text{ km} \times 10\text{ km}$ IoT network and tend to cover the network with 5G/6G edge servers. There are 300 IoT devices in the network, whose distribution follows a unified HetNet model [29] combining Poisson Point Processes and Poisson Cluster Processes to simulate both “uniform” devices and “clustered” devices. The upload transmission rate can vary from 1 Gbps (4 G) to 1 Tbps (6G) [5]. Edge servers are standardized with Intel Xeon E5-2637 (3.0 GHz), storage capacity 5 TB, according to the parameters of Dell PowerEdge R720. We assume edge servers are mounted on vehicles with speed 30 km/h. In this evaluation, we simulate a long-term IoT network by allowing devices to generate tasks with different periodicity and delay requirements. The delay requirements for the delay-tolerant tasks vary from (1,120) minutes. There is no pre-set delay requirement for the delay-sensitive tasks, and we require the servers to process such tasks and return results with best efforts. Table II elaborates the settings of key parameters used in our experiments.

Dataset: To simulate various workloads from tasks, we implement several typical applications on NVIDIA Jetson TX2 to collect task workloads. Here, we use the FLOPS (both CPU and GPU) as computing workloads. Specifically, we execute three computer vision models (YOLOv5s,⁷ ResNet152 [30], EfficientDet-D7⁸) on the 10-minute videos with 30fps sampling rate from DIV2K dataset.⁹ In each video, we resize the frames to different resolutions, i.e., 360 p, 480 p, 540

TABLE III
WORKLOADS (TFLOPS) OF BENCHMARK APPLICATIONS WITH DIFFERENT RESOLUTIONS

[NW]App.Resol.	360p	480p	540p	720p	1080p
YOLOv5	275.2	280.6	281.5	284.9	295.5
ResNet152	199.6	187.7	186.8	184.4	182.2
EfficientDet	160.2	172.1	174.6	175.6	178.1

p, 720 p and 1080 p. The detailed results are shown in Table III.

Benchmarks: We compare ParallEdge and the state-of-the-art works as follows.

- **StaticEdge** [8]: All edge servers are stationary and deployed at the optimal places using the algorithm proposed in [8].
- **MobileEdge** [11]: It is originally designed to use a single mobile server for edge service. For a fair comparison, we modify MobileEdge as a multi-server path planning scheme. For all mobile edge servers, the paths are planned following the framework in [11]. The key difference is that servers in MobileEdge have to finish tasks from devices in the current spot before moving to the next spot.
- **EdgeGO** [31]: It is a server deployment framework which also allows mobile servers to process offloaded tasks during the movement. It does not consider the multi-server scenario or static servers. For fair comparisons, we use area partitioning for EdgeGO’s device-server assignment, following most existing mobile frameworks [10], [11].
- **HybridUAV** [32]: HybridUAV is another hybrid deployment framework, which deploys mobile UAV servers to handle workloads that exceed the static server capacities. As the deployment of static servers is not specified in HybridUAV, we follow the existing work [8] to first deploy the static servers and then use HybridUAV to deploy mobile UAV servers.

B. Performance Evaluation

We first study the deployment cost in terms of the number of required servers. Then we compare the overall delay performance between ParallEdge and MobileEdge given the same number of mobile servers. We also evaluate the computing efficiency of different works to analyze resource utilization and explore the reason for the improvements. Each of the simulations is repeated 100 times to reduce the negative impact of randomness.

Deployment Cost: Fig. 6 shows the performance in terms of deployed servers for full-service coverage. Fig. 6(a) shows the number of required servers under different fractions of delay-sensitive tasks. As each task’s processing time is different, we use the relative extra delay that could be tolerated as a measure of the delay requirement. Specifically, if a task’s deadline is within (100%, 120%) times of its processing time, it is viewed as a delay-sensitive task. Otherwise, the task is treated as a delay-tolerant task. For example, if a task’s processing time is 10 minutes, and its deadline is 11 minutes after the task request, this task is treated as a delay-sensitive task. We change the fraction of

⁶Although the cost of deploying the two types of edge servers (stationary and mobile) is different, the move-nowhere server transformation can still work by adding weights for two different server types. We can compute the deployment costs with movable or in-movable servers and then select the server type with lower deployment costs.

⁷[Online]. Available: <https://github.com/ultralytics/yolov5>

⁸[Online]. Available: <https://github.com/xuannianz/EfficientDet>

⁹[Online]. Available: <https://data.vision.ee.ethz.ch/cvl/DIV2K/>

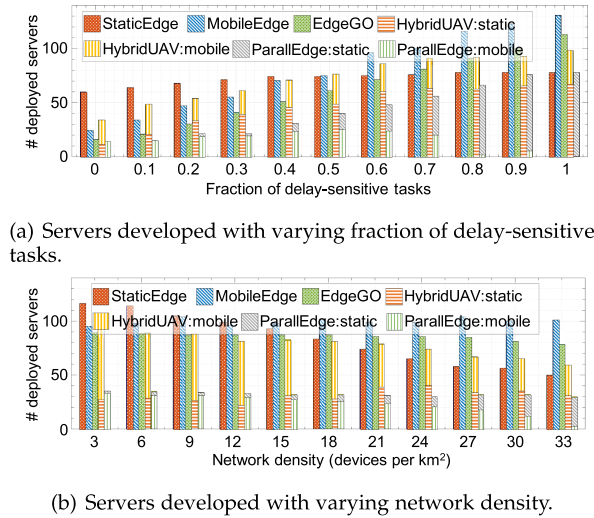


Fig. 6. Deployment cost of 5G/6G edge networks with different frameworks.

delay-sensitive tasks, and explore in which situation ParalleEdge achieves better/worse performance. As shown in the figure, we see that 1) as the fraction of delay-sensitive tasks increases, for all three works, more servers are required to cover all IoT devices with strict deadline requirements. The deployment costs of StaticEdge and ParalleEdge increase slowly since computing resource is inefficiently utilized when delay-tolerant tasks are the majority. 2) For the schemes that utilize mobile servers, the deployment cost becomes larger than that with all static servers (StaticEdge) while ParalleEdge achieves a lower cost than the other schemes. When all tasks are delay-sensitive, ParalleEdge achieves the same performance as StaticEdge. The reason is that ParalleEdge does not separate the deployments of static and mobile servers, instead, it treats all servers as mobile servers and those servers with zero mobility are then treated as static servers. As a result, ParalleEdge can gradually adjust its fractions of mobile and static servers according to the task delay requirements and finally evolves into StaticEdge when all tasks are delay-sensitive. From this experiment, we infer that ParalleEdge is more suitable for delay-tolerant tasks. It is worth noting that, in the 5G/6G edge computing scenarios, the tasks can be very computing-intensive with large data-size (e.g., 8 K video analysis). Although the tasks can be delay-tolerant, it is still impossible for the front-end IoT devices to deal with those tasks.

Fig. 6(b) depicts the number of deployed servers. We fix the number of IoT devices and change the geo-density of the devices (the target area is changed accordingly). We can see that 1) ParalleEdge always uses a minimal number of edge servers to cover all IoT devices. 2) As the network density increases, the number of required servers for all schemes decreases because more IoT devices can be covered by a server in dense networks. 3) ParalleEdge and HybridUAV use both mobile and static servers and thus require fewer servers than other schemes in most cases. ParalleEdge always outperforms HybridUAV, as HybridUAV first uses static servers to cover the IoT devices and then deploys mobile servers to deal with the workloads that

exceed the capacities of the static servers, which causes that: a) some static servers are deployed to cover the sparse areas with few IoT devices and b) the parallelism between moving and processing is overlooked as UAVs are hovering at fixed positions. 4) As the network density increases, fewer mobile servers are used in ParalleEdge and HybridUAV, because a mobile server covers a comparable number of IoT devices with a static server and the cost reduction from server mobility becomes smaller. From this experiment, we see that ParalleEdge achieves more improvements when the network is more large-scale and sparse.

Delay Performance: We further study the performance of ParalleEdge. As the moving delay is not considered in HybridUAV, we mainly compare ParalleEdge with EdgeGO and MobileEdge in terms of the overall delay of task processing for all IoT devices. Fig. 7 shows the comparison in terms of the overall task delay. Recall that a server's task delay is defined as the duration from the time when the server starts moving in the path to the time when all tasks along the path are processed and the results are returned. A smaller delay indicates a better resource utilization of all edge servers.

Fig. 7(a) shows the comparison under different network scales with fixed node density (the number of IoT devices changes accordingly). We observe that 1) ParalleEdge and EdgeGO outperform MobileEdge as they both support processing during mobility and thus can reduce servers' idle time. 2) ParalleEdge achieves shorter overall delay than EdgeGO. This is because ParalleEdge explicitly allocates the IoT devices and servers to fully exploit the "move-while-processing" potential, while EdgeGO uses area partitioning to match IoT devices and servers. Fig. 7(b) depicts the overall delay under different network densities. As expected, ParalleEdge outperforms MobileEdge and EdgeGO. However, we see that the performance gap reduces with the density decreases. The reason is that MobileEdge still has space to cover more IoT devices with high IoT density. Hence, when the density decreases and the area become larger, it is still possible for MobileEdge to cover the same number of IoTs with more reasonable path plans. Differently, for each density setting, ParalleEdge tries the best to minimize the server number, and thus the number is more likely to be increased when the density decreases.

Fig. 7(c) shows the results with varying the average computing workload of IoT devices. When computing workload is light, ParalleEdge has a similar performance to its counterparts since task processing time takes a larger fraction in the overall delay. As a result, there is little space for the computing-moving parallelism. When computing workload is heavy, i.e., computing time and moving time are comparable, computing and moving can be better paralleled and thus a shorter overall delay can be achieved. Fig. 7(d) depicts the overall delay under different transmission rates corresponding to different network conditions. We see that 1) the overall delay of both works decreases as the transmission rate improves. 2) ParalleEdge consistently outperforms its counterparts, especially in low data-rate networks. This is because task uploading is also parallelized with other task processing in ParalleEdge. Therefore, when the transmission rate is high, task uploading takes a small fraction in the overall delay,

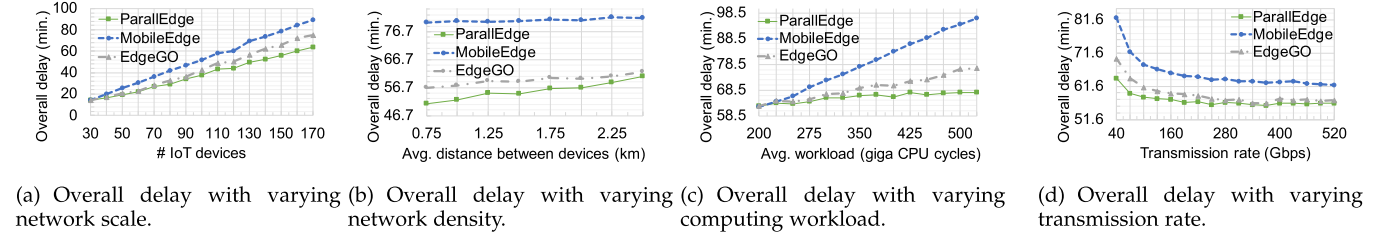


Fig. 7. Performance comparison with MobileEdge and ParalleEdge under various settings.

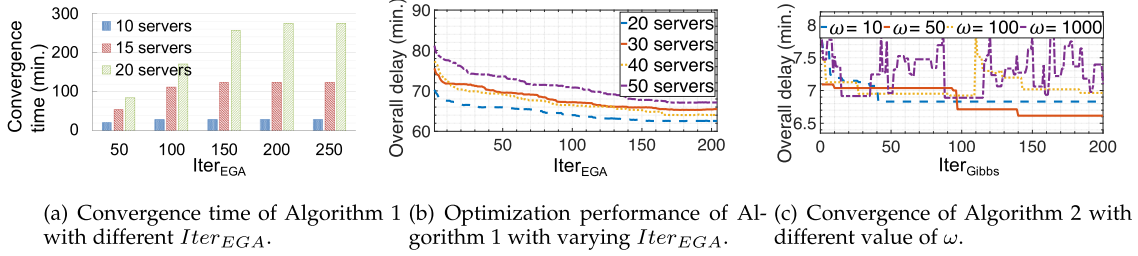


Fig. 8. Algorithm convergence evaluation under different network scales.

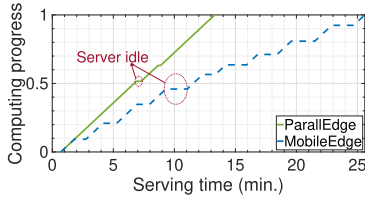


Fig. 9. Computing efficiency during the system serving.

leading to a smaller delay reduction from uploading-moving parallelism.

Computing Efficiency: To explain why ParalleEdge outperforms MobileEdge, we choose a repetitive experiment when ParalleEdge outperforms MobileEdge, and compare the average progress of tasks processing over time, i.e., the ratio of processed tasks to the whole tasks in a given time duration, as shown in Fig. 9. We see that in the experiment, the server idle time, i.e., the time used solely for server moving (horizontal line segments as marked in the figure), is the key to the performance difference. ParalleEdge has much less idle time than MobileEdge due to that the processing time is paralleled with the server's movement.

Algorithm Convergence: We evaluate the convergences of Algorithms 1 and 2, corresponding to multi-server scenarios and single-server scenarios, respectively. Given a 4 km² area with 200 devices, the experiments are implemented on a HP Omen computer with i7-12700 K and NVIDIA RTX 2070Ti. Fig. 8(a) and (b) show the convergence time and optimization performance (i.e., overall processing delay) of Algorithm 1 with different number of servers, respectively. We can see that 1) Algorithm 1 always converges with the significantly large $Iter_{EGA}$, as revealed in Theorem 2. 2) With enlarged

the network scales, the algorithm needs a larger $Iter_{EGA}$ to guarantee convergence. Fig. 8(c) presents the convergence of Algorithm 2 with different values of smoothing factor ω . In general, a small ω will speed convergence yet end to fall into local-optimal solutions. For example, Algorithm 2 converges within 45 iterations with $\omega = 10$, while it converges within 150 iterations with $\omega = 100$. In our experiment, the most appropriate value of ω is 50, which can achieve a satisfying balance between convergence speed and quality of solutions. Thus, we evaluate the convergence of Algorithm 1 with $\omega = 50$. Based on the results of Fig. 8, we set $Iter_{EGA} = 200$ and $Iter_{Gibbs} = 150$ in the experiments.

VI. RELATED WORKS

The existing frameworks on edge deployment can be divided into two categories: static servers and mobile servers.

Static Framework: Most of the existing work on server development follow ETSI standards where *static servers* are connected with base stations or access points (APs) deployed at the logical edge of networks to cover target areas [8], [9]. They aim to combine computation workloads, wireless dynamics and geographical information, etc. to find the best locations for cost-effective server deployment. Zhao et al. [8] proposed a three-phase deployment approach considering both traffic density and wireless density to reduce the number of required edge nodes. This work is used as the benchmark “StaticEdge” in our experiments. For 5G edge networks, Li et al. [9] focused on packet processing and traffic steering, which is the main functions of UPF (User Plane Function) in 5G. Taking locations of UPF in the 5G topology into consideration, the authors proposed an edge server deployment algorithm based on particle swarm optimization.

While these static edge frameworks can reduce the deployment cost or improve QoS to some extent, they neglect the conflict between limited communication range and raising computation requirement. As a result, the number of required servers is still unacceptably high in large-scale networks with massive IoT devices. And the above imperfection will be further highlighted due to the more considerable conflict in 5G/6G networks.

Mobile Framework: Some of the existing work exploit *mobile servers* to reduce the development cost, where edge servers obtain mobilities by mounted on vehicles or unmanned aerial vehicles (UAVs) [10], [11], [12], [33]. For example, Xiang et al. [10] proposed an adaptive cloudlet placement method for mobile applications over GPS Big Data. This work confirmed the locations of mobile cloudlets based on the K-means algorithm, which is used as the device allocation method in our benchmark “MobileEdge”. Some work also use vehicles [10], [11] or UAVs [12], [33] to bring mobilities to edge servers. Since UAVs are more sensitive to energy consumption, these works tend to optimize communication resources for energy saving in mobile server placement. The authors in [33] exploited UAVs as flying base stations to achieve on-demand coverage for ground user devices in 5G and beyond 5G networks. They proposed a hybrid algorithm to minimize the deployment cost of UAVs and maximize the load balance among UAVs.

Similar to ParalleEdge, Liu et al. [11] also used the vehicle-mounted edge servers to serve hot-spots and formulated their joint optimization problem into a mixed-integer nonlinear program, aiming to maximum the number of completed tasks within time deadlines. Specifically, they jointly optimized path planning and communication resource allocation based on the branch & bound algorithm and proposed a low-complexity L-step lookahead branch approach for large-scale networks. The differences between the work [11] and ParalleEdge lie in two aspects: First, ParalleEdge follows the “move-while-processing” paradigm, yet the compared work follows the “move-then-process” paradigm, which is used as the benchmark “MobileEdge” in our evaluation. Second, ParalleEdge focuses on multi-server scenarios involving the sub-problem of device-server allocation, while the compared work only considers the single-server scenario. Even this work can be extended to multiple mobile edges, the sub-problem of device allocation can be solved by area partition algorithms like K-means, since server movement is a waste for resource utilization.

In the end, existing mobile frameworks follow a repetitive “move-then-process” paradigm, assuming that the IoT tasks are lightweight and the computing delay can be neglected compared to the server moving delay. Considering more considerable conflict between communication ranges and computation requirements in upcoming 5G/6G eras, the sojourn time at each spot will be unneglectable with this paradigm and thus deteriorates resource utilization.

VII. CONCLUSION

In this paper, we propose ParalleEdge, a hybrid deployment scheme that employs both static and mobile edge servers for

cost-effective service coverage. By exploiting the computing-mobility parallelism of the mobile servers, fewer servers are required for the same coverage compared to the existing schemes. We also propose a two-stage optimization algorithm with reduced complexity for use in practical scenarios. Extensive experiments are carried out to evaluate the performance of ParalleEdge. The results show that ParalleEdge can drastically reduce both deployment cost and the overall task delay for the service coverage of the 5G/6G edge computing.

VIII. DISCUSSION

ParalleEdge also remains some interesting issues that can be further studied in the future. One is the energy consumption problem, i.e., the consumed energy of each mobile server may be increased in the “move-while-processing” paradigm. The reason is two-fold: 1) the number of required servers is reduced, meaning that covering the same area requires higher mobility for each server. 2) To reduce the overall delay, a mobile server in ParalleEdge prefers moving to another spot and collecting more computation requests during the task processing time instead of waiting for task completion without moving. As a result, longer moving distance per server are required, leading more energy consumption for moving. In the future, we plan to optimize the system performance with the consideration of both energy consumption and the server battery capacity.

The impact of satellite communications (one of the major elements in 6G networks) on our framework is also an important and interesting topic. From the resource perspective, ParalleEdge would relieve the bandwidth constraints of ground-satellite links [34], [35] by allocating massive dispersed devices to the hybrid edge servers. From the coverage perspective, ParalleEdge is mutually complementary to 6G satellite networks. Current satellite networks, like Starlink [36], cannot cover the whole earth all the time, which can be handled by ground-deployed edge servers in ParalleEdge, while some terrain complex areas do not suit for deploying ground edge servers, which can be covered by the server-equipped satellites. In the future, we would study the satellite-ground server cooperation scheme to further improve the system performance in terms of deployment cost and resource utilization.

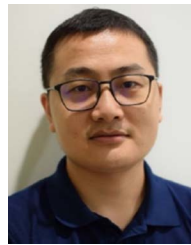
REFERENCES

- [1] S. Jiang, Z. Lin, Y. Li, Y. Shu, and Y. Liu, “Flexible high-resolution object detection on edge devices with tunable latency,” in *Proc. ACM Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 559–572.
- [2] S. Hu, W. Shi, and G. Li, “CEC: A containerized edge computing framework for dynamic resource provisioning,” *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3840–3854, Jul. 2023.
- [3] W. Sun, Y. Zhao, W. Ma, B. Guo, L. Xu, and T. Q. Duong, “Accelerating convergence of federated learning in MEC with dynamic community,” *IEEE Trans. Mobile Comput.*, early access, Feb. 02, 2023, doi: [10.1109/TMC.2023.3241770](https://doi.org/10.1109/TMC.2023.3241770).
- [4] X. Ge, S. Tu, G. Mao, C.-X. Wang, and T. Han, “5G ultra-dense cellular networks,” *IEEE Wireless Commun.*, vol. 23, no. 1, pp. 72–79, Feb. 2016.
- [5] W. Saad, M. Bennis, and M. Chen, “A vision of 6G wireless systems: Applications, trends, technologies, and open research problems,” *IEEE Netw.*, vol. 34, no. 3, pp. 134–142, May/Jun. 2020.
- [6] W. Sun, S. Lian, H. Zhang, and Y. Zhang, “Lightweight digital twin and federated learning with distributed incentive in air-ground 6G networks,” *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1214–1227, May/Jun. 2023.

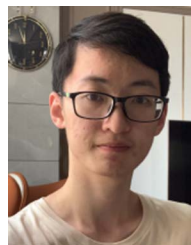
- [7] H. Hejazi and L. Bokor, "A survey on simulation efforts of 4G/LTE-based cellular and hybrid V2X communications," in *Proc. IEEE 44th Int. Conf. Telecommun. Signal Process.*, 2021, pp. 333–339.
- [8] Z. Zhao, G. Min, W. Gao, Y. Wu, H. Duan, and Q. Ni, "Deploying edge computing nodes for large-scale IoT: A diversity aware approach," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3606–3614, Oct. 2018.
- [9] Y. Li, A. Zhou, X. Ma, and S. Wang, "Profit-aware edge server placement," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 55–67, Jan. 2022.
- [10] H. Xiang et al., "An adaptive cloudlet placement method for mobile applications over GPS Big Data," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.
- [11] Y. Liu, Y. Li, Y. Niu, and D. Jin, "Joint optimization of path planning and resource allocation in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 9, pp. 2129–2144, Sep. 2020.
- [12] Z. Ning et al., "5G-enabled UAV-to-community offloading: Joint trajectory design and task scheduling," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3306–3320, Nov. 2021.
- [13] C. Ye, C. Mertz, and M. Satyanarayanan, "Bus on the edge: Continuous monitoring of traffic and infrastructure," U.S. Patent 69A3551747111, Aug. 31, 2021.
- [14] M. Giordani, M. Polese, M. Mezzavilla, S. Rangan, and M. Zorzi, "Toward 6G networks: Use cases and technologies," *IEEE Commun. Mag.*, vol. 58, no. 3, pp. 55–61, Mar. 2020.
- [15] S. Poikonen and B. Golden, "Multi-visit drone routing problem," *Comput. Operations Res.*, vol. 113, 2020, Art. no. 104802.
- [16] V. Ržička and F. Franchetti, "Fast and accurate object detection in high resolution 4K and 8K video using GPUs," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2018, pp. 1–7.
- [17] B. Liu et al., "Multi-agent attention double actor-critic framework for intelligent traffic light control in urban scenarios with hybrid traffic," *IEEE Trans. Mobile Comput.*, early access, Jan. 05, 2023, doi: [10.1109/TMC.2022.3233879](https://doi.org/10.1109/TMC.2022.3233879).
- [18] S. Vatansever, A. E. Dirik, and N. Memon, "Analysis of rolling shutter effect on ENF-based video forensics," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 9, pp. 2262–2275, Sep. 2019.
- [19] Q. Han, S. Yang, X. Ren, P. Zhao, C. Zhao, and Y. Wang, "PCFed: Privacy-enhanced and communication-efficient federated learning for industrial IoTs," *IEEE Trans. Ind. Inform.*, vol. 18, no. 9, pp. 6181–6191, Sep. 2022.
- [20] L. Yu, "Empirical study of python call graph," in *Proc. IEEE/ACM 34th Int. Conf. Automated Softw. Eng.*, 2019, pp. 1274–1276.
- [21] M. Younis and K. Akkaya, "Strategies and techniques for node placement in wireless sensor networks: A survey," *Ad Hoc Netw.*, vol. 6, no. 4, pp. 621–655, 2008.
- [22] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, and M. Protasi, *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*, Berlin, Germany: Springer, 2013.
- [23] S. Li et al., "Enhancing resilience in mobile edge computing under processing uncertainty," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 3, pp. 659–674, Mar. 2023.
- [24] Y. Cui et al., "Software defined cooperative offloading for mobile cloudlets," *IEEE/ACM Trans. Netw.*, vol. 25, no. 3, pp. 1746–1760, Jun. 2017.
- [25] L. Dai, B. Wang, Z. Ding, Z. Wang, S. Chen, and L. Hanzo, "A survey of non-orthogonal multiple access for 5G," *IEEE Commun. Surv. Tut.*, vol. 20, no. 3, pp. 2294–2323, Third Quarter 2018.
- [26] S. Jošilo and G. Dán, "Computation offloading scheduling for periodic tasks in mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 28, no. 2, pp. 667–680, Apr. 2020.
- [27] S. Hougardy, F. Zaiser, and X. Zhong, "The approximation ratio of the 2-opt heuristic for the metric traveling salesman problem," *Operations Res. Lett.*, vol. 48, no. 4, pp. 401–404, 2020.
- [28] M. Chen, S. C. Liew, Z. Shao, and C. Kai, "Markov approximation for combinatorial network optimization," *IEEE Trans. Inf. Theory*, vol. 59, no. 10, pp. 6301–6327, Oct. 2013.
- [29] C. Saha, M. Afshang, and H. S. Dhillon, "3GPP-inspired HetNet model using Poisson cluster process: Sum-product functionals and downlink coverage," *IEEE Trans. Commun.*, vol. 66, no. 5, pp. 2219–2234, May 2018.
- [30] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2017, pp. 126–135.
- [31] R. Cong, Z. Zhao, G. Min, C. Feng, and Y. Jiang, "EdgeGO: A mobile resource-sharing framework for 6G edge computing in massive IoT systems," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 14 521–14 529, Aug. 2022.
- [32] J. Wang, K. Liu, and J. Pan, "Online UAV-mounted edge server dispatching for mobile-to-mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1375–1386, Feb. 2020.
- [33] H. Wang, H. Zhao, W. Wu, J. Xiong, D. Ma, and J. Wei, "Deployment algorithms of flying base stations: 5G and beyond with UAVs," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10 009–10 027, Dec. 2019.
- [34] D. Vasisht, J. Shenoy, and R. Chandra, "L2D2: Low latency distributed downlink for LEO satellites," in *Proc. ACM SIGCOMM Conf.*, 2021, pp. 151–164.
- [35] G. Giuliani, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 317–331.
- [36] J. C. McDowell, "The low earth orbit satellite population and impacts of the spacex starlink constellation," *Astrophysical J. Lett.*, vol. 892, no. 2, 2020, Art. no. L36.
- [37] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2076–2085.



Rong Cong received the BS degree from the Beijing University of Post and Telecommunications, Beijing, China, in 2020. She is currently working toward the PhD degree with the University of Electronic Science and Technology of China. Her research interests include edge computing, mobile computing and Internet of Things. She has published several papers in flagship conferences and journals including IEEE SECON, *IEEE Internet of Things Journal*, etc. She is a reviewer of *IEEE Transactions on Mobile Computing* and *IEEE Internet of Things Journal*.



Zhiwei Zhao (Member, IEEE) received the BS degree from Xi'an Jiaotong University, and the PhD degree from the College of Computer Science, Zhejiang University. He is a professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China (UESTC). His research interests include edge computing and IoT systems, heterogeneous wireless networks, and protocol design. He is a member of ACM, CCF and IEEE.



Linyuanqi Zhang received the BS degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2021. He is currently working toward the MS degree with the Edge Computing and Computer Network, University of Electronic Science and Technology of China. His research interests include Edge computing, low-power IoT, and artificial intelligence.



Geyong Min (Member, IEEE) received the BSc degree in computer science from the Huazhong University of Science and Technology, China, in 1995, and the PhD degree in computing science from the University of Glasgow, U.K., in 2003. He is a professor of high performance computing and networking with the Department of Computer Science, College of Engineering, Mathematics and Physical Sciences, University of Exeter, U.K. His research interests include computer networks, wireless communications, parallel and distributed computing, ubiquitous computing, multimedia systems, and modelling and performance engineering.